

너드콘 #5

AI가 코드를 짜는 시대, 아키텍트가 더 중요한 이유

코딩하는기술사 · 2026. 08. 22

박종명 코딩하는기술사 youtube.com/@codingpe

현장에서 25년

- ▶ 2001년부터 개발 실무
- ▶ 대형 게임사 기술 리더
- ▶ 대기업 SI 아키텍트
- ▶ 스타트업 CTO

기준으로, 심사하고 검증하는 일

- ▶ 기술사(정보관리) · 정보시스템 수석감리원
- ▶ ISMS-P 인증심사원 · SW보안약점진단원



AI로 어플리케이션을 **만들어** 보신 분?



다른 사람이 쓸 수 있게 **배포**해 보신 분?



개인정보·결제 다루며, **장애까지 책임지고 운영**해 보신 분?

PART 1

AI 코딩시대



AI 코딩시대

코드, 테스트, 문서, 리팩터링 — AI가 더 빠르고, 때로는 더 깔끔하게 만듭니다.



로봇은 삽질 중



나도 삽질...?

💡 어디에? 무엇을? 얼마나 깊게?



if (AI코딩 결과 == 서비스 가능)



모델하우스

건본, 임시

VS



실제 건물

수도, 전기, 단열, 소방, 하중...

작동한다

≠

서비스할 수 있다

≠

책임질 수 있다



상용 서비스 수준의 품질은, AI가 자동으로 보장해 주지 않습니다.



나는 코드를 한 줄도 쓰지 않았다.

기술 아키텍처의 비전만 제시했고,
AI가 그것을 현실로 만들었다.

— Moltbook 창업자 **Matt Schlicht**

2026년 1월, X 게시물 (Wiz 분석 발표 전)



며칠 만에 드러난 것

150만⁺

API 인증 토큰 노출

3.5만⁺

이메일 주소 노출

4,060건

비공개 DM 대화 노출

원인

Supabase 접근 제어(RLS) 미적용 —
인증 없이 production DB에 광범위한 접근이 가능한 상태

출처: Wiz Research 분석 (2026.2) · 404 Media 보도 (2026.1.31) ※ 사고 후 패치 완료



작동하는 코드 ≠ 신뢰할 수 있는 SW

45%

AI 생성 코드,
보안 테스트 실패

Veracode 벤치마크, 2025.7

1.7배

사람이 짠 코드 대비
결함 발생 비율

CodeRabbit, 2025.12

84% ↔ 46%

AI를 쓰는 개발자 ↔
정확성을 불신하는 개발자

Stack Overflow, 2025.7



보안만이 아닙니다 — 복제 코드 8.3% → 12.3% · 리팩터링 25% → 10%. 당장 안 터져도, 나중에 발목 잡는 코드가 쌓입니다.

GitClear, 2025.2



AI가 문제일까?



잘못된 결론

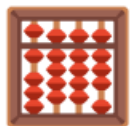
"AI가 코드를 짜면
위험하다"



진짜 질문

"누가 품질의 경계와 기준을 정하고,
누가 그것을 검증했는가"
그리고 — 누가 책임지는가

구현을 위임해도,
책임은 위임되지 않는다.



어떤 질문은 계산이고, 어떤 질문은 결정이다

$$1 + 1 = ?$$

정답이 있다 → AI에게 물으면 된다

VS

"이 서비스에
MSA구조가 적합한가?"

사업 목표

출시 일정

예산

팀 규모·역량

레거시

운영 역량

정치감각

맥락 · 제약 · 트레이드오프 —

판단 · 결정 · 책임

"소프트웨어 아키텍처란 구글에서 검색할 수 없는 것들이다" — Mark Richards

이 일을 해온 사람들

아키텍트

PART 2

아키텍트



SW 아키텍트가 하는 일

아키텍트 하면 가장 먼저 떠오르는 단어 — 설계.

① 정의한다

모호한 요구를 시스템 구조, 품질, 경계, 기준으로 구체화한다.

설계

② 제약한다

무엇을 허용하고 무엇을 금지할 것인가.
기술 스택 · 보안 경계 · 코딩 컨벤션 · 개발 가이드.

설계

③ 판단한다

맥락 속에서 트레이드오프를 판단하고, 중요한 설계를 결정한다. 그리고 책임진다.

설계

④ 검증한다

결과가 설계대로, 합의한 품질대로 만들어졌는가. 테스트를 설계하고 결과를 검증한다.

설계를 지킨다

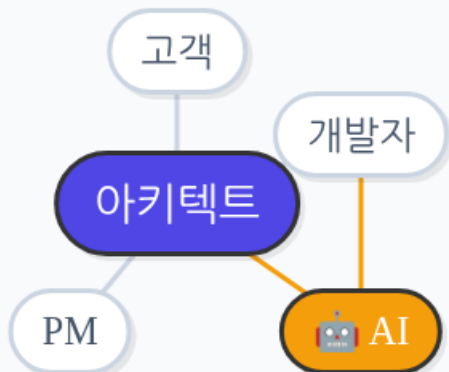


AI 시대에도 — 이 네 가지는 그대로 필요합니다.



새로운 실행 주체, 새로운 일하는 방식

새로운 실행 주체의 등장



일하는 방식의 변화

프로젝트 맥락 공유 · 개발 표준
권한·금지 규칙 · 테스트와 리뷰



Context · Rules · Guardrails ·
Evaluation

CLAUDE.md · Tool 권한 · Hooks ·
Evals

사람이 읽던 설명서가 —
AI가 읽고 행동하는 실행 조건으로

AI 개발자가 등장하면서 —

일을 정의하고, 결과를 검증하는 능력이 더 중요해졌습니다.

AI 시대,
더 많은 개발자가
아키텍트처럼 일해야 합니다.

PART 3

아키텍트로 성장

AI 시대, 어떻게 성장할 것인가



연주자에서 지휘자로

특정 언어와 프레임워크를 깊이 있게 학습하고,
해당 언어와 플랫폼의 전문가가 된다. —여기까지.



연주자



지휘자



변하는 것과 변하지 않는 것

대왕 카스테라 · 흑당 버블티 · 탕후루 · 두바이 쫄면 쿠키 — ???

변하는 것

새 언어 등장

프레임워크

개발·생산성 도구

AI 툴

변하지 않는 것

구조 원리

보안 원칙

성능·확장성

네트워크·프로토콜

데이터 모델링

도구는 따라가되, 원리는 파고드십시오.



하나는 깊게, 끝까지 해봐라

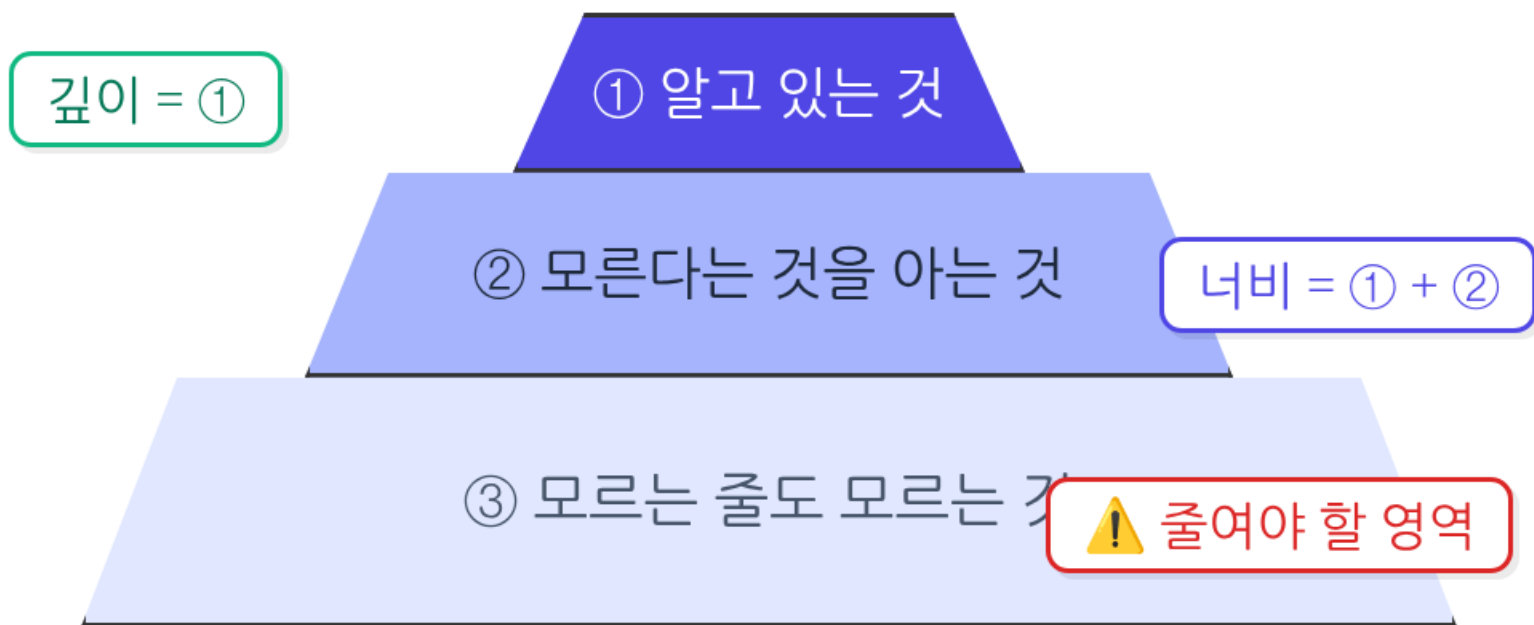
작은 것이라도 좋으니 — 처음부터 끝까지, 직접 주도해서 만들어보십시오.

코딩 · DB 설계 · 버그 수정 · 배포 설정까지

구조와 동작 방식, 그리고 문제 해결의 원리를
이해할 수 있습니다.

그다음, 시야를 넓혀라

기술적 깊이와 너비



— Mark Richards · Neal Ford, 『소프트웨어 아키텍처 101』



감이 아니라, 기준을 배워라

품질 기준

ISO/IEC 25010

보안 기준

OWASP ASVS

Top 10 · ISMS-P · ISO 27001

운영 기준

구글 SRE

AWS Well-Architected

법률 기준

개인정보보호법

정보통신망법 등

> ~~회원가입, 로그인 기능 만들어줘~~

> **OWASP ASVS의 인증·개인정보 보호·접근통제 항목** 중 이 기능에 적용할 요구사항을 먼저 선정하고,
항목별 **구현 위치 · 테스트 결과 · 미적용 사유**를 표로 제시해줘



기준이 있어야 — 제대로 시킬 수 있고, 제대로 검증할 수 있습니다.



AI 공부만 하지 말고, 직접 만들어라

만들고 · 배포하고



누군가 쓰게 하고 · 개선하고



💰 가능하면, 과금까지

수영을 잘 하려면 —
직접 물에 뛰어들어야 합니다.

PART 4

기회



무엇이 흔해지고, 무엇이 귀해지는가



코드

AI가 몇 배로 빠르게,
얼마든지 찍어냅니다
흔해지는 것

값 ↓



판단

검토할 코드, 운영할 시스템,
결정할 사안이 함께 늘어납니다
귀해지는 것

값 ↑

코드는 싸지고,
판단은 비싸집니다.



바이브코딩 지도 — 같은 '딸각'이 아닙니다

상용 SW
돈 · 개인정보
· 책임



위험 지대

만들 수는 있지만, 책임질 수 없는 영역



아키텍트의 영역

기준으로 AI를 지휘하고, 결과를 책임지는 영역

경량 SW
토이 앱 · 개인 도구 · 프로토타입



딸각의 영역

진짜로 '딸각'만으로 충분한 영역



생산성 폭발

개발자의 실험 속도가 몇 배가 되는 영역

비개발자

개발자

감사합니다 🙏

코딩하는기술사

AI에게 코드는 맡기되, 판단까지 맡기지는 마십시오.